



MEDS

Maktab-e-Digital Systems

Department of Electrical Engineering • UET Lahore

MODULE 2

C Language for Hardware Engineers

Summer Training Programme 2026 • Cohort 4

Week 2 of 8-Week Intensive Training

Prepared by: Umer Shahid, Shehzeen Malik, & Aman Murad

Version 1.0 • April 2026



Table of Contents

Table of Contents.....	2
1. Module Overview.....	4
2. Learning Objectives.....	5
3. Weekly Schedule.....	7
4. Day 1 — C Foundations: Compilation, Types & Control Flow.....	8
4.1 The C Compilation Pipeline.....	8
4.2 Data Types & Fixed-Width Integers.....	9
4.3 Bitwise Operations.....	9
4.4 Control Flow.....	11
4.5 Functions.....	12
4.6 Day 1 Exercises.....	12
5. Day 2 — Pointers, Arrays & Memory Layout.....	13
5.1 Memory Layout of a C Program.....	13
5.2 Pointers — The Heart of C.....	13
5.3 Arrays.....	15
5.4 Strings in C.....	15
5.5 Functions with Pointers (Pass-by-Reference).....	16
5.6 Day 2 Exercises.....	16
6. Day 3 — Structs, Unions, Enums & the Hardware Connection.....	18
6.1 Structs — Grouping Related Data.....	18
6.2 Enums — Named Constants.....	19
6.3 Unions — Same Memory, Different Views.....	20
6.4 Putting It All Together: RISC-V CPU State.....	21
6.5 Day 3 Exercises.....	21
7. Day 4 — Dynamic Memory, File I/O & Debugging.....	23
7.1 Dynamic Memory Allocation.....	23
7.2 File I/O.....	24
7.3 Command-Line Arguments in C.....	25
7.4 Debugging with GDB.....	26
7.5 Memory Checking with Valgrind.....	26
7.6 Day 4 Exercises.....	27
8. Day 5 — Preprocessor, Multi-File Projects & Grand Assignment.....	28
8.1 The C Preprocessor.....	28
8.2 Header Files & Include Guards.....	29
8.3 Multi-File Project Organization.....	30
8.4 Makefile for Multi-File C Project.....	30
8.5 Day 5 Exercises.....	32



9. Grand Assignment: RISC-V Instruction Decoder.....	33
9.1 Project Description.....	33
9.2 Repository Structure.....	33
9.3 Requirements.....	33
9.4 Expected Output.....	35
9.5 Bonus Challenges (+10 points).....	35
10. Recommended Resources.....	36
Primary References.....	36
RISC-V Specific.....	36
Tools.....	36
Practice.....	36
11. Appendix: C Quick Reference & Common Pitfalls.....	37
A. printf Format Specifiers for Hardware Work.....	37
B. Common Pitfalls.....	37
C. RISC-V RV32I Encoding Quick Reference.....	38



1. Module Overview

Module 2 takes you into C — the language that sits closest to the hardware. Every RISC-V simulator, every EDA tool backend, every firmware stack, and every embedded system is written in C (or its close relatives). Understanding C is not about learning another programming language; it is about understanding how a computer actually works at the machine level.

This module focuses on the aspects of C that matter most for hardware engineers: memory layout, pointers, bitwise operations, structs that map to hardware registers, and low-level patterns you will encounter when writing testbenches, firmware, and architecture simulators. We deliberately skip topics like GUI programming or web development — everything here connects to your future work with RISC-V.

Why C for Hardware Engineers?

When you write a RISC-V compliance test, the reference model is in C. When you write firmware for an FPGA prototype, it is in C. When you read the RISC-V Spike simulator source code, it is C/C++. When you interface with Verilator through DPI, you write C. C is not just a language — it is the bridge between your RTL design and the real world.

Hardware Connection

Throughout this module, look for these purple boxes. They connect each C concept to a specific hardware engineering application, showing you exactly why you need to know this.

Pre-requisites: Module 1 completed. You must be comfortable with the terminal, Git, and Makefiles.

Compiler: GCC (gcc) on Linux. All code compiled with: `gcc -Wall -Wextra -std=c11 -g`

Deliverable: Grand Assignment — a RISC-V instruction decoder written in C, submitted as a Git repository.



2. Learning Objectives

By the end of this module, every mentee should be able to:

Compilation & Fundamentals

- Explain the 4 stages of C compilation: preprocessing, compilation, assembly, linking
- Use gcc flags for warnings (-Wall -Wextra), debugging (-g), optimization (-O2), and standards (-std=c11)
- Understand and use fixed-width integer types (uint32_t, int8_t) from <stdint.h>
- Perform bitwise operations (AND, OR, XOR, shifts) for bit-field extraction and manipulation
- Write clean, readable code with proper formatting, naming conventions, and comments

Pointers & Memory

- Explain the difference between stack, heap, data, and text segments
- Declare and use pointers, including pointers to pointers and void pointers
- Understand pointer arithmetic and its relationship to array indexing
- Use arrays (1D and 2D) and understand their memory layout (row-major)
- Work with C strings (null-terminated char arrays) and common string functions
- Pass arrays and pointers to functions; understand pass-by-value vs. pass-by-reference

Structs & Data Modeling

- Define and use structs to model hardware registers, instruction formats, and CSRs
- Understand struct memory layout, padding, and alignment
- Use unions for type-punning (reinterpreting the same memory as different types)
- Use enums for readable opcode and control signal definitions
- Combine bit-fields, unions, and structs to decode RISC-V instructions

Dynamic Memory & I/O

- Allocate and free memory with malloc, calloc, realloc, and free
- Detect and fix memory leaks, dangling pointers, and buffer overflows
- Read and write files using fopen, fclose, fprintf, fscanf, fgets
- Parse binary and hex data files (e.g., reading a .hex memory image)

Preprocessor & Multi-File Projects

- Use #define for constants, macros, and conditional compilation



- Write and use header files with proper include guards (`#ifndef/#define/#endif`)
- Organize multi-file C projects with separate `.c` and `.h` files
- Write Makefiles with dependency tracking for multi-file projects
- Use GDB for basic debugging: breakpoints, stepping, printing variables, backtrace



3. Weekly Schedule

This module is spread across 5 working days.

Day	Topics	Mode
Day 1	C Foundations: Compilation pipeline, data types, fixed-width integers, operators, bitwise operations, control flow	Lab
Day 2	Pointers, Arrays & Memory: Stack/heap layout, pointer arithmetic, arrays, strings, pass-by-reference, const correctness	Lab
Day 3	Structs, Unions, Enums: Hardware register modeling, bit-fields, padding/alignment, RISC-V instruction encoding	Lab
Day 4	Dynamic Memory, File I/O & Debugging: malloc/free, file parsing, hex file reading, GDB, Valgrind basics	Lab
Day 5	Preprocessor, Multi-File Projects & Grand Assignment: Macros, header files, include guards, linking, project start	Lab

Grand Assignment Completion: Due 48 hours after Day 5 session (submitted as GitHub repo)



4. Day 1 — C Foundations: Compilation, Types & Control Flow

Goal: Understand how C code becomes machine code, master fixed-width types, and perform bitwise operations essential for hardware work.

4.1 The C Compilation Pipeline

Unlike interpreted languages (Python), C code goes through 4 distinct stages before it runs. Understanding this pipeline demystifies many errors you will encounter:

1. Preprocessing (cpp): Expands `#include`, `#define`, `#ifdef`. Output: still C code, but expanded.
2. Compilation (cc1): Translates C to assembly (.s file). This is where syntax and type errors appear.
3. Assembly (as): Translates assembly to machine code object file (.o). Platform-specific binary.
4. Linking (ld): Combines object files + libraries into a final executable. Resolves function calls across files.

```
# See each stage explicitly:
```

```
gcc -E main.c -o main.i      # Step 1: Preprocessor output
gcc -S main.c -o main.s      # Step 2: Assembly output
gcc -c main.c -o main.o      # Step 3: Object file
gcc main.o -o main           # Step 4: Link into executable
```

```
# Or all at once (what you normally do):
```

```
gcc -Wall -Wextra -std=c11 -g -o main main.c
```

```
# Flag meanings:
```

```
# -Wall      Enable most warnings
# -Wextra    Enable extra warnings
# -std=c11   Use C11 standard
# -g         Include debug info (for GDB)
# -O0        No optimization (default, best for debugging)
# -O2        Optimize for speed (production builds)
```

Hardware Connection: Assembly Output



Run `gcc -S` on a simple C function and examine the `.s` file. You will see RISC-V assembly instructions (if cross-compiling) or x86 assembly. This is exactly what your RISC-V processor will execute. In Weeks 3–4, you will read and write this assembly directly.

4.2 Data Types & Fixed-Width Integers

In hardware engineering, you need exact control over data sizes. The standard C types (`int`, `long`) have platform-dependent sizes. Instead, always use fixed-width types from `<stdint.h>`:

Type	Size	Range	Use Case
<code>uint8_t</code>	1 byte	0–255	Byte-level data, register bytes
<code>int8_t</code>	1 byte	-128–127	Signed byte values
<code>uint16_t</code>	2 bytes	0–65535	Half-word values, CSR fields
<code>int16_t</code>	2 bytes	±32767	Signed half-word
<code>uint32_t</code>	4 bytes	0–4.29B	RV32 registers, instructions, addresses
<code>int32_t</code>	4 bytes	±2.14B	Signed 32-bit values
<code>uint64_t</code>	8 bytes	0–18.4E	RV64 registers, addresses
<code>int64_t</code>	8 bytes	±9.2E	Signed 64-bit values
<code>size_t</code>	4/8 bytes	Unsigned	Array indices, memory sizes
<code>bool</code>	1 byte	0 or 1	Flags (<code>#include <stdbool.h></code>)

Never Use `int` for Hardware Work

The size of `int` is compiler-dependent (could be 16, 32, or 64 bits). When modeling a 32-bit RISC-V register, use `uint32_t` — always. This is non-negotiable in embedded and hardware code.

4.3 Bitwise Operations

Bitwise operations are the most hardware-relevant skill in C. Every instruction decode, every register read, every CSR manipulation uses bitwise ops:

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint32_t a = 0xDEADBEEF;
    uint32_t b = 0xFF00FF00;

    // Basic operations
```



```

printf("AND:  0x%08X\n", a & b);    // Mask bits
printf("OR:   0x%08X\n", a | b);    // Set bits
printf("XOR:  0x%08X\n", a ^ b);    // Toggle bits
printf("NOT:  0x%08X\n", ~a);       // Invert all bits
printf("SHL:  0x%08X\n", a << 4);    // Shift left 4
printf("SHR:  0x%08X\n", a >> 4);    // Shift right 4

// ---- RISC-V Instruction Decoding Example ----
// A RISC-V instruction is 32 bits wide.
// The opcode is bits [6:0]
uint32_t instruction = 0x00A28233;    // add x4, x5, x10

uint32_t opcode = instruction & 0x7F;    // bits [6:0]
uint32_t rd      = (instruction >> 7) & 0x1F;    // bits [11:7]
uint32_t funct3  = (instruction >> 12) & 0x07;    // bits [14:12]
uint32_t rs1     = (instruction >> 15) & 0x1F;    // bits [19:15]
uint32_t rs2     = (instruction >> 20) & 0x1F;    // bits [24:20]
uint32_t funct7  = (instruction >> 25) & 0x7F;    // bits [31:25]

printf("opcode=0x%02X rd=x%u funct3=%u ", opcode, rd, funct3);
printf("rs1=x%u rs2=x%u funct7=0x%02X\n", rs1, rs2, funct7);

return 0;
}

```

Common Bit Manipulation Patterns

```

// Extract bits [high:low] from value
#define EXTRACT_BITS(val, high, low) \
    (((val) >> (low)) & ((1U << ((high)-(low)+1)) - 1))

// Set bit N
#define SET_BIT(val, n)    ((val) | (1U << (n)))

// Clear bit N
#define CLEAR_BIT(val, n)  ((val) & ~(1U << (n)))

// Toggle bit N
#define TOGGLE_BIT(val, n) ((val) ^ (1U << (n)))

```



```
// Check if bit N is set
#define IS_BIT_SET(val, n) (((val) >> (n)) & 1U)

// Sign-extend a value from bit_width to 32 bits
int32_t sign_extend(uint32_t val, int bit_width) {
    uint32_t sign_bit = 1U << (bit_width - 1);
    return (int32_t)((val ^ sign_bit) - sign_bit);
}
```

Hardware Connection: Instruction Decoding

The bit extraction code above is EXACTLY what your RISC-V processor does in hardware. In Verilog, you write `assign opcode = instruction[6:0]`. In C, you write `(instruction & 0x7F)`. Same operation, different syntax. Mastering this in C prepares you to think like the hardware.

4.4 Control Flow

Quick revision of control flow since you know this from prior courses, with emphasis on hardware-relevant patterns:

```
// switch-case: PERFECT for opcode decoding
switch (opcode) {
    case 0x33: // R-type
        execute_r_type(instruction);
        break;
    case 0x13: // I-type (immediate)
        execute_i_type(instruction);
        break;
    case 0x23: // S-type (store)
        execute_s_type(instruction);
        break;
    case 0x63: // B-type (branch)
        execute_b_type(instruction);
        break;
    default:
        fprintf(stderr, "Unknown opcode: 0x%02X\n", opcode);
        break;
}

// Ternary operator: useful for mux-like selections
uint32_t result = (opcode == 0x33) ? alu_result : immediate;
```



4.5 Functions

```
// Function declaration (prototype) – goes in header file
uint32_t decode_opcode(uint32_t instruction);
void print_register(uint32_t reg_num, uint32_t value);

// Function definition
uint32_t decode_opcode(uint32_t instruction) {
    return instruction & 0x7F;
}

// Void return: function performs an action, returns nothing
void print_register(uint32_t reg_num, uint32_t value) {
    printf("x%u = 0x%08X (%d)\n", reg_num, value, (int32_t)value);
}
```

4.6 Day 1 Exercises

Day 1 Hands-On Exercises

- ▶ Exercise 1: Write a program that takes a 32-bit hex value as a command-line argument and prints it in binary, decimal (signed & unsigned), and hex. Use strtoul() for parsing.
- ▶ Exercise 2: Compile a simple program through all 4 stages separately (-E, -S, -c, link). Examine each output file. How large is the .s file compared to the .c file?
- ▶ Exercise 3: Write a function uint32_t extract_field(uint32_t instruction, int high, int low) that extracts bits [high:low]. Test it by decoding the R-type instruction 0x00A28233 (add x4, x5, x10).
- ▶ Exercise 4: Write a program that takes an RV32 instruction as hex input and prints all fields: opcode, rd, funct3, rs1, rs2, funct7. Test with at least 5 different instructions.
- ▶ Exercise 5: Implement and test sign_extend(). Verify: sign_extend(0xFFFF, 12) should produce -1 (0xFFFFFFFF as int32_t).
- ▶ Exercise 6 (Bonus): Write a function that packs rd, rs1, rs2, funct3, funct7, and opcode back into a 32-bit R-type instruction. Verify by decoding your packed result.

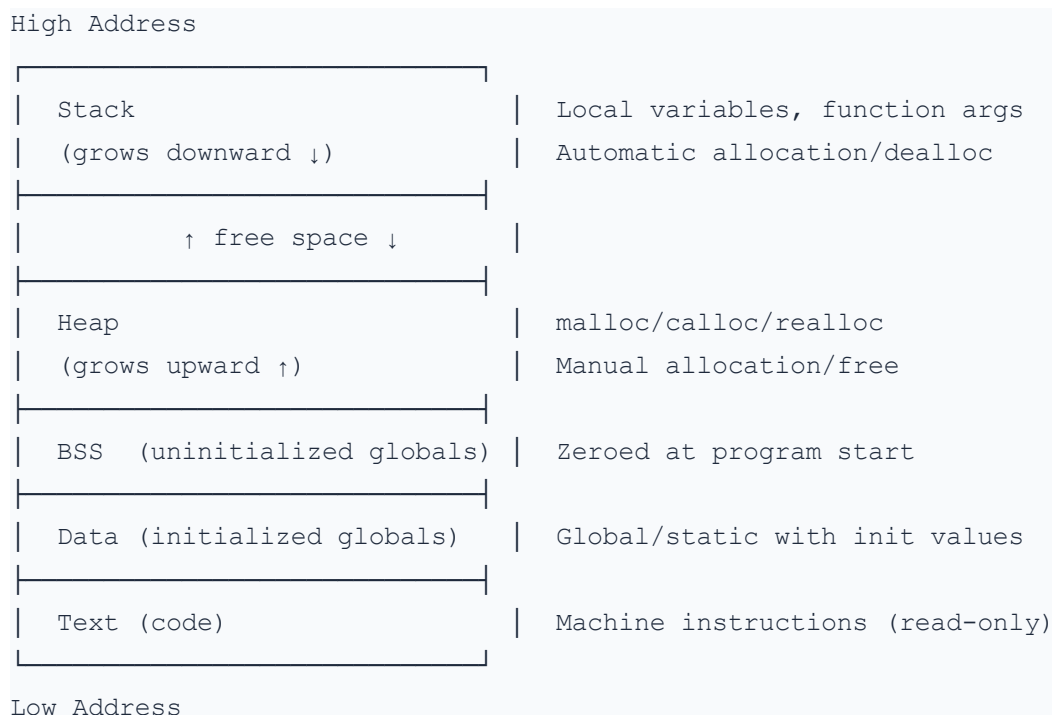


5. Day 2 — Pointers, Arrays & Memory Layout

Goal: Understand how C organizes memory, master pointers as the fundamental mechanism for hardware-style data access, and work with arrays and strings.

5.1 Memory Layout of a C Program

When your program runs, the operating system gives it a virtual address space divided into segments:



Hardware Connection: Address Space

This is exactly the memory map of your RISC-V processor! When you write a linker script for an FPGA SoC, you place `.text` at ROM addresses, `.data/.bss` at RAM addresses, and the stack pointer at the top of RAM. Understanding this diagram is understanding your processor's memory system.

5.2 Pointers — The Heart of C

A pointer is a variable that holds a memory address. Every pointer has two attributes: the address it stores, and the type of data it points to (which determines how many bytes to read/write when dereferenced).

```
#include <stdio.h>
#include <stdint.h>
```



```
int main(void) {
    uint32_t register_value = 0xDEADBEEF;
    uint32_t *ptr = &register_value; // ptr holds ADDRESS of
    register_value

    // Dereference: access the value at the address
    printf("Value: 0x%08X\n", *ptr);          // 0xDEADBEEF
    printf("Address: %p\n", (void *)ptr);      // Some hex address
    printf("Size of pointer: %zu bytes\n", sizeof(ptr)); // 8 on 64-bit

    // Modify through pointer
    *ptr = 0xCAFEBADE;
    printf("New value: 0x%08X\n", register_value); // 0xCAFEBADE!

    // NULL pointer: points nowhere (safe default)
    uint32_t *null_ptr = NULL;
    // *null_ptr = 42; // CRASH! Segfault. Never dereference NULL.

    return 0;
}
```

Pointer Arithmetic

When you add 1 to a pointer, it advances by `sizeof(*ptr)` bytes, not 1 byte. This is why pointer type matters:

```
uint32_t memory[8] = {0}; // Simulated 32-byte memory
uint32_t *word_ptr = memory;

// word_ptr + 1 moves forward 4 bytes (sizeof(uint32_t))
// word_ptr + 2 moves forward 8 bytes
*(word_ptr + 3) = 0xFF; // Same as memory[3] = 0xFF

// Byte-level access to the same memory:
uint8_t *byte_ptr = (uint8_t *)memory;
// byte_ptr + 1 moves forward 1 byte
*(byte_ptr + 12) = 0xAB; // Writes to byte 12 = word 3, byte 0
```

Hardware Connection: Memory-Mapped I/O



In embedded RISC-V systems, hardware peripherals are accessed by reading/writing specific memory addresses. You use volatile pointers: `volatile uint32_t *uart_data = (volatile uint32_t *)0x10000000`; This is pointer arithmetic applied to real hardware registers.

5.3 Arrays

```
// Arrays are contiguous blocks of memory
uint32_t registers[32] = {0}; // 32 RISC-V registers, initialized to 0
registers[0] = 0; // x0 is always 0 in RISC-V
registers[2] = 0x7FFFFFFF0; // sp (stack pointer)

// Array name IS a pointer to the first element
uint32_t *reg_ptr = registers; // No & needed!
// registers[i] is equivalent to *(registers + i)

// 2D array: memory as rows x columns
uint8_t cache[4][64]; // 4 cache lines, 64 bytes each
cache[0][0] = 0xFF; // First byte of first line

// Array size calculation
size_t num_regs = sizeof(registers) / sizeof(registers[0]); // 32
```

Arrays Have No Bounds Checking!

C does NOT check array bounds. Writing `registers[32]` will silently corrupt adjacent memory. This is the source of countless bugs and security vulnerabilities. Always validate indices.

5.4 Strings in C

C strings are simply null-terminated arrays of `char`. The null terminator `\0` (value `0x00`) marks the end of the string:

```
#include <string.h>

// String literals are read-only
char *name = "RISC-V"; // Pointer to read-only string
char mnemonic[] = "ADD"; // Mutable copy on stack

// Common string functions:
size_t len = strlen(mnemonic); // 3 (not counting \0)
strcmp("ADD", "SUB"); // < 0 (ADD < SUB)
```



```
strcpy(dest, src); // Copy (DANGEROUS: no size check)
strncpy(dest, src, sizeof(dest) - 1); // Safer: limited copy
snprintf(buf, sizeof(buf), "x%u", rd); // Formatted, safe
```

5.5 Functions with Pointers (Pass-by-Reference)

// C is ALWAYS pass-by-value. To modify the caller's data, pass a pointer.

// Swap two register values

```
void swap_registers(uint32_t *a, uint32_t *b) {
    uint32_t temp = *a;
    *a = *b;
    *b = temp;
}
```

// Passing an array (decays to pointer automatically)

```
void clear_registers(uint32_t regs[], size_t count) {
    for (size_t i = 0; i < count; i++) {
        regs[i] = 0;
    }
}
```

// const pointer: promise not to modify data through this pointer

```
void print_memory(const uint8_t *mem, size_t size) {
    for (size_t i = 0; i < size; i++) {
        printf("%02X ", mem[i]);
    }
}
```

5.6 Day 2 Exercises

Day 2 Hands-On Exercises

- ▶ Exercise 1: Write a program that declares variables in different segments (local, global, static, malloc'd). Print their addresses and verify they match the memory layout diagram.
- ▶ Exercise 2: Implement a simulated 32-register file as `uint32_t regs[32]`. Write functions: `void write_reg(uint32_t *regs, uint8_t rd, uint32_t value)` that enforces `x0=0`, and `uint32_t read_reg(const uint32_t *regs, uint8_t rs)`.



- ▶ Exercise 3: Write a function `void memory_dump(const uint8_t *mem, size_t size)` that prints memory as a hex dump with addresses (like `xxd`). Format: `0x0000: DE AD BE EF CA FE BA BE |.....|`
- ▶ Exercise 4: Write a function that reverses an array of `uint32_t` in-place using pointer arithmetic (no array indexing `[]`). Test with `{1, 2, 3, 4, 5}`.
- ▶ Exercise 5: Implement `strcat_safe(char *dest, size_t dest_size, const char *src)` that concatenates strings safely without buffer overflow. Return `-1` on error.
- ▶ Exercise 6 (Bonus): Write a simulated 256-byte memory (`uint8_t mem[256]`). Implement `load_word(mem, addr)` and `store_word(mem, addr, value)` that handle byte-addressed access with proper alignment checks.



6. Day 3 — Structs, Unions, Enums & the Hardware Connection

Goal: Model hardware data structures in C using structs, unions, and enums. Decode RISC-V instructions using structured types.

6.1 Structs — Grouping Related Data

Structs let you group related variables into a single type — perfect for modeling hardware structures:

```
// Model a RISC-V decoded instruction
typedef struct {
    uint32_t opcode;
    uint32_t rd;
    uint32_t funct3;
    uint32_t rs1;
    uint32_t rs2;
    uint32_t funct7;
    int32_t imm;           // Sign-extended immediate
} decoded_instr_t;

// Create and use
decoded_instr_t instr;
instr.opcode = 0x33;
instr.rd = 4;
instr.rs1 = 5;
instr.rs2 = 10;

// Pointer to struct (very common pattern)
decoded_instr_t *p = &instr;
p->opcode = 0x13; // Arrow operator: shorthand for (*p).opcode
```

Struct Memory Layout & Padding

The compiler inserts padding bytes between struct members to satisfy alignment requirements. This affects struct size:

```
struct example {
    uint8_t a;           // 1 byte, then 3 bytes padding
    uint32_t b;          // 4 bytes (must be 4-byte aligned)
    uint8_t c;           // 1 byte, then 3 bytes padding
};
```



```
// sizeof(struct example) = 12, not 6!

// Reorder to minimize padding:
struct example_packed {
    uint32_t b;    // 4 bytes
    uint8_t  a;    // 1 byte
    uint8_t  c;    // 1 byte + 2 bytes padding
};

// sizeof(struct example_packed) = 8
```

Hardware Connection: Register Definition

When you write device drivers for RISC-V peripherals, you define a struct that maps directly to the peripheral's register layout. The struct's field offsets must match the hardware register addresses. Padding awareness is critical — a wrong offset means you read the wrong register.

6.2 Enums — Named Constants

```
// RISC-V opcodes as an enum
typedef enum {
    OP_R_TYPE = 0x33, // Register-register
    OP_I_TYPE = 0x13, // Immediate arithmetic
    OP_LOAD   = 0x03, // Load from memory
    OP_STORE  = 0x23, // Store to memory
    OP_BRANCH = 0x63, // Conditional branch
    OP_JAL    = 0x6F, // Jump and link
    OP_JALR   = 0x67, // Jump and link register
    OP_LUI    = 0x37, // Load upper immediate
    OP_AUIPC  = 0x17, // Add upper immediate to PC
    OP_SYSTEM = 0x73, // System instructions (ECALL, CSR)
} opcode_t;

// ALU operations
typedef enum {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR,
    ALU_SLL, ALU_SRL, ALU_SRA, ALU_SLT, ALU_SLTU
} alu_op_t;

// Use with switch for clean dispatch
switch ((opcode_t)opcode) {
    case OP_R_TYPE: /* ... */ break;
```



```

    case OP_LOAD:    /* ... */ break;
    // ...
}

```

6.3 Unions — Same Memory, Different Views

A union stores all members at the same memory address. Only one member is valid at a time. The union's size equals its largest member:

```

// View the same 32-bit instruction as different formats
typedef union {
    uint32_t raw;           // Full 32-bit word
    struct {                // R-type fields
        uint32_t opcode : 7;
        uint32_t rd     : 5;
        uint32_t funct3 : 3;
        uint32_t rs1    : 5;
        uint32_t rs2    : 5;
        uint32_t funct7 : 7;
    } r_type;
    struct {                // I-type fields
        uint32_t opcode : 7;
        uint32_t rd     : 5;
        uint32_t funct3 : 3;
        uint32_t rs1    : 5;
        uint32_t imm     : 12;
    } i_type;
} instruction_t;

// Usage:
instruction_t inst;
inst.raw = 0x00A28233;           // Load full instruction
printf("rd = %u\n", inst.r_type.rd); // Access as R-type
printf("rs1 = %u\n", inst.r_type.rs1);

```

Bit-field Portability Warning

The layout of bit-fields (order, padding) is implementation-defined and varies between compilers and endianness. For portable code, use explicit shift-and-mask operations. Bit-field structs are great for readability and debugging but should not be relied upon for cross-platform correctness.

6.4 Putting It All Together: RISC-V CPU State



```
// Complete RISC-V RV32I CPU state model
typedef struct {
    uint32_t x[32];          // 32 general-purpose registers
    uint32_t pc;             // Program counter
    uint8_t *memory;         // Pointer to memory (allocated on heap)
    size_t mem_size;         // Memory size in bytes

    // Statistics
    uint64_t instr_count;    // Instructions executed
    uint64_t cycle_count;    // Cycles simulated
} cpu_state_t;

// Initialize CPU
void cpu_init(cpu_state_t *cpu, size_t mem_size) {
    memset(cpu->x, 0, sizeof(cpu->x)); // All regs = 0
    cpu->pc = 0x00000000;
    cpu->memory = calloc(mem_size, 1); // Zero-initialized memory
    cpu->mem_size = mem_size;
    cpu->instr_count = 0;
    cpu->cycle_count = 0;
}

// Write register (enforcing x0 = 0)
void reg_write(cpu_state_t *cpu, uint8_t rd, uint32_t value) {
    if (rd != 0) {
        cpu->x[rd] = value;
    }
}
```

6.5 Day 3 Exercises

Day 3 Hands-On Exercises

- ▶ Exercise 1: Define a `decoded_instr_t` struct. Write a function `decode_r_type(uint32_t raw, decoded_instr_t *out)` that fills all fields. Test with `0x00A28233` (add `x4`, `x5`, `x10`).
- ▶ Exercise 2: Define the `opcode_t` enum with all RV32I opcodes. Write `const char *opcode_to_string(opcode_t op)` that returns the mnemonic. Test with all opcodes.



- ▶ Exercise 3: Create a `cpu_state_t` struct. Write `cpu_init()`, `reg_write()`, `reg_read()`. Write `dump_registers()` that prints all 32 registers with their ABI names (`x0/zero`, `x1/ra`, `x2/sp`, etc.).
- ▶ Exercise 4: Create a struct with mixed types and predict its size by hand accounting for padding. Verify with `sizeof()`. Reorder fields to minimize size.
- ▶ Exercise 5: Use a union to create an `instruction_t` with both raw access and R-type/I-type struct views. Decode `0x00500113` (`addi x2, x0, 5`) using the I-type view.
- ▶ Exercise 6 (Bonus): Model a RISC-V UART peripheral as a struct with control, status, `tx_data`, and `rx_data` registers. Write `uart_putchar()` and `uart_getchar()` functions that read/write through the struct.



7. Day 4 — Dynamic Memory, File I/O & Debugging

Goal: Allocate memory dynamically, read/write files (including binary hex files), and debug programs with GDB and Valgrind.

7.1 Dynamic Memory Allocation

```
#include <stdlib.h>
#include <string.h>

// malloc: allocate uninitialized memory
uint32_t *memory = malloc(4096 * sizeof(uint32_t));
if (memory == NULL) {
    fprintf(stderr, "Memory allocation failed!\n");
    exit(EXIT_FAILURE);
}

// calloc: allocate AND zero-initialize (preferred for arrays)
uint8_t *mem = calloc(65536, sizeof(uint8_t)); // 64KB zero memory

// realloc: resize existing allocation
memory = realloc(memory, 8192 * sizeof(uint32_t));

// free: ALWAYS free when done (exactly once!)
free(memory);
memory = NULL; // Good practice: NULL after free

free(mem);
mem = NULL;
```

The Four Deadly Memory Sins

5. Memory Leak: malloc without matching free. Memory grows until crash.
6. Dangling Pointer: using a pointer after free. Random data or crash.
7. Double Free: calling free() twice on same pointer. Heap corruption.
8. Buffer Overflow: writing past the allocated boundary. Corrupts other data.

Memory Bugs Are Silent Killers



Unlike array-out-of-bounds in Java or Python, C memory errors often do NOT crash immediately. They silently corrupt data and cause failures much later, making them extremely hard to debug. This is why Valgrind exists.

7.2 File I/O

Text Files

```
// Reading a text file line by line
FILE *fp = fopen("test_results.log", "r");
if (fp == NULL) {
    perror("Failed to open file"); // Prints system error message
    return -1;
}

char line[256];
while (fgets(line, sizeof(line), fp) != NULL) {
    if (strstr(line, "FAIL") != NULL) {
        printf("Failed: %s", line);
    }
}
fclose(fp); // Always close files!

// Writing to a file
FILE *out = fopen("report.txt", "w"); // "w" = overwrite, "a" = append
fprintf(out, "Test Results:\n");
fprintf(out, "Pass: %d, Fail: %d\n", pass_count, fail_count);
fclose(out);
```

Binary / Hex Files

A common task is loading a hex memory image into your simulated memory:

```
// Sample hex file (one 32-bit word per line, in hex):
// 00500113
// 00A00193
// 003100B3

int load_hex_file(const char *filename, uint8_t *memory, size_t mem_size)
{
    FILE *fp = fopen(filename, "r");
    if (!fp) {
```



```

        perror("Cannot open hex file");
        return -1;
    }

    char line[32];
    uint32_t addr = 0;
    while (fgets(line, sizeof(line), fp) && addr + 3 < mem_size) {
        uint32_t word = (uint32_t)strtoul(line, NULL, 16);
        // Store as little-endian (RISC-V convention)
        memory[addr + 0] = (word >> 0) & 0xFF;
        memory[addr + 1] = (word >> 8) & 0xFF;
        memory[addr + 2] = (word >> 16) & 0xFF;
        memory[addr + 3] = (word >> 24) & 0xFF;
        addr += 4;
    }
    fclose(fp);
    return (int)(addr / 4); // Number of words loaded
}

```

Hardware Connection: Memory Image Loading

This is EXACTLY how RTL simulators load memory. In Verilog, you use `$readmemh("program.hex", memory)`. In your C simulator, you write `load_hex_file()`. Both do the same thing: read hex values from a file into a simulated memory array.

7.3 Command-Line Arguments in C

```

int main(int argc, char *argv[]) {
    // argc: argument count (including program name)
    // argv[0]: program name
    // argv[1]: first argument, etc.

    if (argc < 2) {
        fprintf(stderr, "Usage: %s <hex_file> [options]\n", argv[0]);
        return EXIT_FAILURE;
    }

    const char *hex_file = argv[1];
    int verbose = 0;

    for (int i = 2; i < argc; i++) {

```



```

        if (strcmp(argv[i], "--verbose") == 0 || strcmp(argv[i], "-v") ==
0) {
            verbose = 1;
        } else if (strcmp(argv[i], "--mem-size") == 0 && i + 1 < argc) {
            // ... parse next argument ...
            i++; // Skip value argument
        }
    }

    printf("Loading: %s (verbose=%d)\n", hex_file, verbose);
    return EXIT_SUCCESS;
}

```

7.4 Debugging with GDB

```

# Compile with debug info
gcc -g -O0 -o decoder decoder.c

# Start GDB
gdb ./decoder

# Essential GDB commands:
(gdb) break main           # Set breakpoint at main()
(gdb) break decoder.c:42   # Breakpoint at line 42
(gdb) run program.hex      # Run with arguments
(gdb) next                 # Step over (execute line)
(gdb) step                 # Step into function call
(gdb) print variable       # Print variable value
(gdb) print/x register_val # Print in hex
(gdb) print cpu.x[5]       # Print struct member
(gdb) backtrace            # Show call stack
(gdb) continue             # Continue to next breakpoint
(gdb) watch variable       # Break when variable changes
(gdb) info locals          # Print all local variables
(gdb) quit                 # Exit GDB

```

7.5 Memory Checking with Valgrind

```

# Run program under Valgrind
valgrind --leak-check=full ./decoder program.hex

```



```
# What Valgrind reports:
# - Memory leaks (malloc without free)
# - Use of uninitialized values
# - Invalid memory reads/writes
# - Double frees
# - Buffer overflows

# A clean run should show:
# All heap blocks were freed -- no leaks are possible
# ERROR SUMMARY: 0 errors from 0 contexts
```

7.6 Day 4 Exercises

Day 4 Hands-On Exercises

- ▶ Exercise 1: Write a program that allocates a 64KB memory array, loads a hex file into it, dumps the first 64 bytes, and frees the memory. Run with Valgrind to verify zero leaks.
- ▶ Exercise 2: Intentionally create each of the 4 memory sins (leak, dangling, double-free, overflow) in separate test files. Run each with Valgrind and observe the error messages.
- ▶ Exercise 3: Write a program that reads a simulation log file and counts PASS/FAIL/SKIP. Print a summary. Handle file-not-found gracefully. (Reuse log format from Module 1.)
- ▶ Exercise 4: Implement `load_hex_file()` and store results as little-endian. Verify by loading 0xDEADBEEF and checking individual bytes: EF, BE, AD, DE.
- ▶ Exercise 5: Debug a provided buggy program using GDB. Set breakpoints, step through, print variables, find and fix 3 bugs.
- ▶ Exercise 6 (Bonus): Write a program that takes `--mem-size`, `--start-addr`, and `--trace` flags from the command line. Parse them properly and use them to configure a memory simulation.



8. Day 5 — Preprocessor, Multi-File Projects & Grand Assignment

Goal: Organize production-quality multi-file C projects with proper headers, macros, and build systems. Begin the Grand Assignment.

8.1 The C Preprocessor

Constants & Macros

```
// Constants (prefer this over magic numbers)
#define NUM_REGISTERS    32
#define MEMORY_SIZE      65536    // 64 KB
#define PC_START         0x00000000
#define OPCODE_MASK      0x7F

// Function-like macros (use parentheses everywhere!)
#define EXTRACT_BITS(val, high, low) \
    (((val) >> (low)) & ((1U << ((high) - (low) + 1)) - 1))

#define MAX(a, b)    ((a) > (b) ? (a) : (b))
#define MIN(a, b)    ((a) < (b) ? (a) : (b))

// Debug logging macro (compiled out in release builds)
#ifdef DEBUG
#define LOG(fmt, ...) \
    fprintf(stderr, "[%s:%d] " fmt "\n", __FILE__, __LINE__, \
    ##__VA_ARGS__)
#else
#define LOG(fmt, ...)    // No-op in release
#endif
```

Conditional Compilation

```
// Support both RV32 and RV64 with compile-time switch
#ifdef RV64
    typedef uint64_t reg_t;
    #define REG_FMT "0x%016lX"
#else
    typedef uint32_t reg_t;
    #define REG_FMT "0x%08X"
#endif
```



```
// Compile with: gcc -DRV64 ... for 64-bit mode
// Compile with: gcc -DDEBUG ... for debug logging
```

8.2 Header Files & Include Guards

```
// ===== cpu.h =====
#ifndef CPU_H
#define CPU_H

#include <stdint.h>
#include <stddef.h>

// Type definitions
typedef struct {
    uint32_t x[32];
    uint32_t pc;
    uint8_t *memory;
    size_t mem_size;
} cpu_state_t;

// Function declarations (prototypes only)
void    cpu_init(cpu_state_t *cpu, size_t mem_size);
void    cpu_destroy(cpu_state_t *cpu);
uint32_t cpu_fetch(cpu_state_t *cpu);
int     cpu_execute(cpu_state_t *cpu, uint32_t instruction);
void    cpu_dump_regs(const cpu_state_t *cpu);

#endif // CPU_H

// ===== cpu.c =====
#include "cpu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void cpu_init(cpu_state_t *cpu, size_t mem_size) {
    memset(cpu->x, 0, sizeof(cpu->x));
    cpu->pc = 0;
    cpu->memory = calloc(mem_size, 1);
```



```

    if (!cpu->memory) {
        fprintf(stderr, "Failed to allocate %zu bytes\n", mem_size);
        exit(EXIT_FAILURE);
    }
    cpu->mem_size = mem_size;
}

void cpu_destroy(cpu_state_t *cpu) {
    free(cpu->memory);
    cpu->memory = NULL;
}

```

8.3 Multi-File Project Organization

A well-organized project separates concerns into modules. Here is the recommended structure:

```

riscv-decoder/
├─ Makefile
├─ include/
│   └─ cpu.h           # CPU state + function prototypes
│   └─ decoder.h       # Instruction decoding
│   └─ memory.h        # Memory subsystem
│   └─ common.h        # Shared types, macros, constants
├─ src/
│   └─ main.c          # Entry point, argument parsing
│   └─ cpu.c           # CPU implementation
│   └─ decoder.c       # Instruction decoder logic
│   └─ memory.c        # Memory load/store operations
├─ test/
│   └─ test_decoder.c  # Decoder unit tests
│   └─ test_programs/  # Hex files for testing
│       └─ add.hex
│       └─ branch.hex
└─ docs/
    └─ README.md

```

8.4 Makefile for Multi-File C Project

```

CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -g -Iinclude
LD_FLAGS =

```



```
SRC_DIR    = src
OBJ_DIR    = build
BIN_DIR    = bin
TEST_DIR   = test

SRC        = $(wildcard $(SRC_DIR)/*.c)
OBJ        = $(SRC:$(SRC_DIR)/%.c=$(OBJ_DIR)/%.o)
TARGET     = $(BIN_DIR)/riscv-decoder

.PHONY: all clean test debug release help

all: dirs $(TARGET)

dirs:
@mkdir -p $(OBJ_DIR) $(BIN_DIR)

$(TARGET): $(OBJ)
$(CC) $(LDFLAGS) -o $@ $^
@echo "Build complete: $@"

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
$(CC) $(CFLAGS) -c $< -o $@

# Automatic dependency generation
$(OBJ_DIR)/%.d: $(SRC_DIR)/%.c | dirs
$(CC) $(CFLAGS) -MM -MT $(OBJ_DIR)/$*.o $< > $@

-include $(OBJ:.o=.d)

debug: CFLAGS += -DDEBUG -O0
debug: all

release: CFLAGS += -O2 -DNDEBUG
release: all

test: all
./$(TARGET) test/test_programs/add.hex

valgrind: all
```



```
valgrind --leak-check=full ./$(TARGET) test/test_programs/add.hex

clean:
rm -rf $(OBJ_DIR) $(BIN_DIR)

help:
@echo "Targets: all, debug, release, test, valgrind, clean, help"
```

8.5 Day 5 Exercises

Day 5 Hands-On Exercises (1 hour — remaining time for Grand Assignment)

- ▶ Exercise 1: Split a single-file decoder program into 3 files (main.c, decoder.c, decoder.h). Write a Makefile that compiles them. Verify it builds correctly.
- ▶ Exercise 2: Add conditional compilation for RV32 vs RV64 using `#ifdef`. Add a debug macro that prints to stderr when `-DDEBUG` is passed.
- ▶ Exercise 3: Create proper include guards for all header files. Test by `#including` the same header twice and verifying no errors.



9. Grand Assignment: RISC-V Instruction Decoder

GRAND ASSIGNMENT

MEDS Module 2 — Due: 48 Hours After Day 5

Submission: GitHub repository link sent to your mentor

9.1 Project Description

You will build a RISC-V RV32I Instruction Decoder — a command-line tool that reads a hex file containing RISC-V machine code, decodes each instruction, and prints the human-readable assembly. This is the front-end of a CPU simulator and directly prepares you for Weeks 3–4 (RISC-V ISA).

9.2 Repository Structure

```
riscv-decoder/
├── README.md
├── Makefile
├── .gitignore
├── include/
│   ├── common.h      # Shared macros, types, constants
│   ├── decoder.h     # Decoder function prototypes & types
│   └── memory.h       # Memory subsystem prototypes
├── src/
│   ├── main.c        # Entry point, CLI parsing
│   ├── decoder.c     # Instruction decode logic
│   └── memory.c       # Hex file loading & memory ops
├── test/
│   ├── test_decoder.c # Unit tests for decoder
│   └── programs/      # Test hex files
│       ├── r_type.hex # R-type test cases
│       ├── i_type.hex # I-type test cases
│       ├── branch.hex # Branch test cases
│       └── mixed.hex  # Mixed instruction program
└── docs/
    └── DESIGN.md      # Design decisions & decoder logic
```



9.3 Requirements

Part A: Decoder Logic (40 points)

Your decoder must correctly decode the following RV32I instruction types:

- R-type: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
- I-type (arithmetic): ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI
- I-type (load): LB, LH, LW, LBU, LHU
- S-type (store): SB, SH, SW
- B-type (branch): BEQ, BNE, BLT, BGE, BLTU, BGEU
- U-type: LUI, AUIPC
- J-type: JAL
- I-type (jump): JALR

For each instruction, output the decoded assembly in this format:

```
0x00000000: 00500113  addi  x2, x0, 5
0x00000004: 00A00193  addi  x3, x0, 10
0x00000008: 003100B3  add   x1, x2, x3
0x0000000C: 0020A023  sw    x2, 0(x1)
0x00000010: 00108063  beq   x1, x1, 0
0x00000014: DEADBEEF  UNKNOWN
```

Requirements:

- All immediates must be properly sign-extended
- Unknown instructions must print UNKNOWN (not crash)
- Use enums for opcodes and ALU operations
- Use structs for decoded instruction representation

Part B: Project Structure (25 points)

- Minimum 3 source files (main.c, decoder.c, memory.c) with corresponding headers
- Proper include guards on all header files
- Use #define for all constants (no magic numbers in code)
- At least one function-like macro (e.g., EXTRACT_BITS)
- Makefile with all, clean, test, debug, valgrind targets
- Zero memory leaks (verified by Valgrind)

Part C: Git Workflow (20 points)

- Minimum 12 meaningful commits with descriptive messages



- At least 2 feature branches (e.g., feature/r-type-decode, feature/load-hex)
- At least 1 merge or rebase into main
- Proper .gitignore (build/, bin/, *.o, *.d files excluded)

Part D: Testing & Documentation (15 points)

- At least 4 hex test files covering all instruction types
- README.md with: project description, build instructions, usage examples, sample output
- DESIGN.md explaining your decoder's structure and key design decisions
- Inline comments on non-obvious code (especially bit manipulation)

9.4 Expected Output

```
$ ./bin/riscv-decoder test/programs/mixed.hex
RISC-V RV32I Instruction Decoder
=====
Loaded 8 instructions from test/programs/mixed.hex
```

Addr	Hex	Assembly
0x00000000	00500113	addi x2, x0, 5
0x00000004	00A00193	addi x3, x0, 10
0x00000008	003100B3	add x1, x2, x3
0x0000000C	40310133	sub x2, x2, x3
0x00000010	0020A023	sw x2, 0(x1)
0x00000014	0000A103	lw x2, 0(x1)
0x00000018	FE209CE3	bne x1, x2, -8
0x0000001C	004000EF	jal x1, 4

```
Decoded 8 instructions (8 valid, 0 unknown)
```



10. Recommended Resources

Primary References

- The C Programming Language, 2nd Ed. (K&R) — Chapters 1–6, the definitive C reference
- CS:APP (Computer Systems: A Programmer's Perspective) — Chapters 2–3 on data representation and machine code
- MIT Missing Semester, Lecture 7: Debugging & Profiling — <https://missing.csail.mit.edu/>
- Beej's Guide to C Programming (free) — <https://beej.us/guide/bgc/>

RISC-V Specific

- RISC-V Unprivileged Spec, Chapter 2 (RV32I) — Instruction encoding tables you will decode
- RISC-V Reference Card — <https://github.com/jameslzh/riscv-card>
- Venus RISC-V Simulator (web) — <https://venus.cs61c.org/> — Verify your decoder output against this

Tools

- GDB Tutorial: <https://sourceware.org/gdb/current/onlinedocs/gdb/>
- Valgrind Quick Start: <https://valgrind.org/docs/manual/quick-start.html>
- cdecl.org — Translate complex C declarations into English

Practice

- Exercism C Track: <https://exercism.org/tracks/c> — C programming exercises
- Project Euler (first 10 problems in C) — Practice algorithms with C
- CS61C Labs (UC Berkeley) — RISC-V C programming assignments



11. Appendix: C Quick Reference & Common Pitfalls

A. printf Format Specifiers for Hardware Work

```
%d    int (signed decimal)
%u    unsigned int
%x    unsigned hex (lowercase)
%X    unsigned hex (uppercase)
%08X  unsigned hex, 8 digits, zero-padded    // 0x0000001F
%p    pointer address
%zu    size_t
%ld    long
%lu    unsigned long
%lx    unsigned long hex
PRIu32, PRIx32  from <inttypes.h> for uint32_t portability
```

B. Common Pitfalls

1. Assignment vs. Comparison

```
if (x = 5)    // BUG! This ASSIGNS 5 to x, always true
if (x == 5)   // CORRECT comparison
// Tip: compile with -Wall to catch this (gcc warns)
```

2. Integer Overflow

```
uint8_t a = 255;
a = a + 1;    // a is now 0 (wraps around) – silent!
int8_t b = 127;
b = b + 1;    // UNDEFINED BEHAVIOR for signed overflow
```

3. Signed vs. Unsigned Comparison

```
int a = -1;
unsigned int b = 1;
if (a < b)    // FALSE! -1 is converted to ~4 billion unsigned
// Always use explicit casts when mixing signed/unsigned
```

4. Array Decay

```
void func(uint32_t arr[]) {
    sizeof(arr);    // Returns sizeof(pointer) = 8, NOT array size!
    // Always pass array size as separate parameter
}
```



5. String Mistakes

```
char s1[] = "hello";    // 6 bytes (5 chars + \0)
char *s2 = "hello";    // Pointer to read-only string
s2[0] = 'H';           // CRASH! String literals are read-only
s1[0] = 'H';           // OK - s1 is a mutable copy
```

6. Forgetting break in switch

```
switch (x) {
    case 1: do_a(); // Falls through to case 2!
    case 2: do_b(); // Always executes after case 1
    // Fix: add break; after each case
}
```

C. RISC-V RV32I Encoding Quick Reference

```
R-type: [funct7(7) | rs2(5) | rs1(5) | funct3(3) | rd(5) | opcode(7)]
I-type: [imm[11:0](12) | rs1(5) | funct3(3) | rd(5) | opcode(7)]
S-type: [imm[11:5](7) | rs2(5) | rs1(5) | funct3(3) | imm[4:0](5) |
opcode(7)]
B-type:
[imm[12|10:5](7) | rs2(5) | rs1(5) | funct3(3) | imm[4:1|11](5) | opcode(7)]
U-type: [imm[31:12](20) | rd(5) | opcode(7)]
J-type: [imm[20|10:1|11|19:12](20) | rd(5) | opcode(7)]
```

End of Module 2

MEDS Lab • UET Lahore • Summer 2026

"C does not hide the hardware — it IS the hardware, in text form."